

Resume



안녕하세요 이영민입니다.

Email

bbok4yo@gmail.com

Channel

[Github](#)

[Velog](#)

ABOUT ME

- React와 Next.js App Router로 동적 라우팅과 전역 상태 관리를 포함한 화면 아키텍처를 설계합니다.
- TypeScript로 타입을 정의하고 컴포넌트 재사용 구조와 API 호출 시점을 서비스 관점에서 설계합니다.
- 백엔드 데이터 구조와 API 연동을 이해하고 프론트엔드 상태 관리 전략을 함께 설계합니다.
- Git과 PR 기반 협업에서 기획 의도를 확인하고 코드 리뷰에 참여합니다.
- AI와 협업해 요구사항 분석부터 SDD 설계, 자동화된 코드 품질 검증까지 개발 흐름을 연결합니다.

CAREERS

리얼라인

기간: 2026.01 ~ 2026.02

- AI 기반 소프트웨어 엔지니어링 파이프라인 구축: 요구사항 분석·SDD 설계·구현·품질 검증 전 과정에 Claude Code를 붙여 개발 주기를 줄였습니다.
- 단기 다중 프로젝트 완수: 1개월 내 전략 보고서 생성기·지도 데이터 시각화·스팟업 웹 3종을 스펙 기반 AI 워크플로우로 구현했습니다.

성과: AI를 활용한 일관된 개발 및 품질 엔지니어링

1. AI-Assisted SDD 개발 및 문서화

- 아키텍처 설계: 요구사항을 바탕으로 Claude Code와 상호작용하며 시스템 아키텍처, 데이터 모델링, 컴포넌트 계층을 정의하는 SDD를 신속하게 작성했습니다.
- 문서화 자동화: API 명세 변경과 상태 관리 플로우를 AI로 문서에 실시간 반영해 최신 명세를 바로 확인하도록 했습니다.

2. Custom Skill·Hook 기반 코드 품질 엔지니어링

- 자동화 품질 검증: Custom Skill과 Git Hook(Pre-commit/Pre-push)을 연동하여 커밋 단계에서 AI가 정적 분석·컨벤션 검사를 자동 수행하도록 구성했습니다.
- 지속적 리팩토링: AI가 프로젝트 전체 컨텍스트를 바탕으로 보안 취약점 점검, 중복 코드 제거, 아키텍처 개선안을 제안하고 적용했습니다.

라운피플

기간: 2024.01 ~ 2024.04

성과: 테스트 케이스(TC) 및 주어진 테스트 항목에 기반한 기능 테스트 수행

- JIRA를 활용해 버그 리포트를 생성하고 이슈를 트래킹했습니다.
- 테스트 결과 보고서를 작성하고 개발팀에 피드백을 전달했습니다.
- 관련 기술: JIRA

ACTIVITIES

프론트엔드 개발자 과정

소속/기관: Kakao x goorm

활동 연도: 2024.07~2025.04

- Next.js를 학습하고 실제 프로젝트에 적용해 실전 감각을 키웠습니다.
- 디자이너, PM, 백엔드 개발자 등 다양한 직군과 협업하면서 직군별 의사결정 우선순위 차이를 좁히는 커뮤니케이션 방식을 익혔습니다.

캠퍼스 SW 아카데미 TABA 빅데이터 분석 및 인공지능 처리를 위한 SW융합개발자양성

소속/기관: 과학기술정보통신부

활동 연도: 2023

- 프론트엔드·백엔드 개발 및 Figma 기반의 기획 등 전반적인 개발 협업 과정을 경험했습니다.
- 협업 톨과 개발 전반을 아우르며 실제 프로젝트에서의 기본적인 협업 프로세스를 이해했습니다.
- 과정 수료 후 프로젝트 부분에서 우수상 및 우수교육생으로 선정되었습니다.

EDUCATION

- 단국대학교 컴퓨터공학과 : 2019.03. ~ 2026.02. (졸업)

STACKS

Front-End

- HTML, CSS(SCSS), JS(ES6) & TS
- React.js, Next.js
- tailwind
- Redux Toolkit, react-query, zustand
- D3.js

Collaboration & Tools

- slack
- Figma

- Git, Github

PROJECTS

MindGraph (AI 지식 캡처 & 그래프 시각화)

서비스 개요: ChatGPT·Gemini·Claude·Grok 4개 LLM 답변을 캡처해 의미 단위로 묶고 D3.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. 기획·설계·개발·QA·내부 검증을 1인이 담당했고 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

팀원: 1명 (1인 PO·풀스택 개발, 출시 후 응대 대비 인프라 사전 구축 중)

기술 스택: TypeScript, Next.js 16, React 19, D3.js, Claude Code, Chrome Extension Manifest V3, IndexedDB, Supabase, GitHub Actions

- 4 LLM의 잦은 UI 변경에 단일 파일 수정으로 대응하기 위해 답변 DOM 선택자를 detector.ts SELECTORS에 모으고 hostname 자동 감지로 통합 레이어를 만들었습니다.
- 1인 운영이 가능하도록 Claude Code 위에 9개 혹·plan·build·qa·ship 4 phase /sprint·4중 SSOT 자동 동기를 설계해 sprint 라이프사이클 반복 작업을 자동화했습니다.
- LLM 응답 정확도와 응대 속도를 사전 확보하기 위해 7부서·9에이전트별 docs를 분리하고 task_type·코드 path를 must-read doc에 매핑해 워커 위임 시 합집합 5개 내외 doc만 첨부되도록 했습니다.
- 출시 후 사용자 응대 사이클을 단일 동선으로 묶기 위해 PostHog·Sentry·OpenTelemetry·api/feedback·api/error-log 5개 채널을 코드 베이스에 사전 구축했습니다.
- 카페·지하철 환경에서 캡처가 유실되지 않도록 IndexedDB 우선 기록과 Pending Ops 큐(MAX_RETRIES=5) 기반 Write-Behind 동기화로 오프라인 CRUD와 온라인 복귀 자동 flush를 구현했습니다.
- MV3 환경의 Extension 카트 데이터(저장한 LLM 답변 아이템) 릴레이를 허가되지 않은 origin이 가져갈 수 없도록 bridge.ts ALLOWED_ORIGINS 화이트리스트로 postMessage origin을 검증했습니다.
- Supabase RLS 정책 직접 설계와 GitHub Actions CI/CD·next-intl 한·영 다국어 운영까지 1인이 담당했습니다.

chatGraph (AI 대화 시각화 플랫폼)

서비스 개요: LLM과의 대화를 꼬리물기 형태의 마인드맵 그래프로 시각화하는 웹 서비스입니다. LLM 응답 대기 동안 발생하는 사용자 흐름 단절을 줄이고 시각화 라이프사이클을 통합하는 클라이언트 책임을 맡았습니다.

팀원: 4명 (FE 2명, BE 2명)

기술 스택: TypeScript, React, Next.js, D3.js, Zustand, Tailwind CSS, TanStack Query, Vitest, LLM API (서버 경유)

- 새 토픽 생성 시 LLM 응답 대기로 화면이 멈추는 문제를 해결하고자 sessionStorage와 temp ID로 즉시 라우팅 후 성공 시 실제 ID로 교체하는 Optimistic 라우팅을 설계해 대기 중에도 채팅 화면을 곧바로 띄웠습니다.
- 사이드바 토픽 클릭 시 초기 로딩 지연을 줄이고자 Zustand 스토어에 LLM 응답을 프리패치하고 useSuspenseQuery의 initialData로 주입해 라우트 전환 시 빈 화면 없이 데이터를 즉시 그렸습니다.
- 팀원이 도입한 D3.js Force Simulation이 리렌더 시 SVG 트리와 충돌하는 문제를 useEffect cleanup과 `d3.selectAll("*").remove()` 로 해결해 데이터 변경 시 SVG 재구성을 보강했습니다.
- 확장에 대비해 Feature-First 구조로 features·views·shared를 분리하고, 843줄까지 비대해진 use-question-tree 혹은 책임 단위로 나눠 355줄로 축소했습니다.
- 페이지 컴포넌트의 책임을 분산하고자 TopicContentLayout·StandardChatView·OptimisticChatView로 뷰 계층을 분리하고 Suspense 경계와 컴포지션을 재설계했습니다.

- findPathToNode 경로 탐색·Zustand 토픽 토글·새 토픽 시작 혹은 Vitest 회귀 테스트를 작성해 핵심 흐름의 회귀를 자동 감지했습니다.